

Static pre-slicing for LLM contract auditing

A controlled A/B experiment measuring whether a purely lexical pre-filter can cut model input by half without degrading agreement with the DIVE ground-truth labels.

Corpus DIVE (22,330 contracts), stratified n = 50 Model Claude Opus 5, effort = high Date 3 September 2026

ABSTRACT

Classifying Solidity source against the DASP-8 taxonomy with a large language model costs tokens roughly in proportion to source length, and most of a real contract is boilerplate that cannot carry a vulnerability. We built `slice.py`, a static pre-filter that keeps only the lines matching lexical anchors for each DASP category, collapses unmodified standard-library constructs, and groups the survivors into blocks tagged with their candidate categories. The filter is validated to retain evidence for **119 of 119** label-positive category instances (recall 1.00 for all eight categories) while emitting **60.2%** of the original characters.

We then ran the 50-contract evaluation twice under identical conditions, varying only the pre-filter. Total token consumption fell **56.6%** (913,958 → 396,438) and wall clock fell **19.7%**, while label agreement was statistically indistinguishable between arms (61.8% vs 62.5%; McNemar on 400 paired cells, $p = 0.76$). We therefore report **non-inferiority at roughly half the input budget**, not an accuracy improvement. Two secondary findings qualify the result: monetary cost fell only 1.4% because the saving lands on the cheapest token class, and the category tags emitted by the filter measurably bias the model toward positive predictions.

TOKENS	AGREEMENT	WALL CLOCK	COST
-56.6%	+0.7 pp	-19.7%	-1.4%
913,958 → 396,438	$p = 0.76$, n.s.	1,019.6 s → 818.5 s	\$3.4103 → \$3.3636

1 Motivation and research question

Our baseline protocol submits each contract's complete source to the model in a single prompt and asks for eight binary labels. On the 50-contract set this consumed 913,958 tokens and roughly 17 minutes. Scaling this to the full DIVE corpus of 22,330 contracts is prohibitive on both budget and context-window grounds, and the marginal value of most of the input is questionable: a fixed-supply ERC-20 token is 400 lines of OpenZeppelin boilerplate wrapped around perhaps 30 lines that could plausibly hold a flaw.

This suggests a pre-processing step. The research question is deliberately narrow:

RESEARCH QUESTION

Can a purely static, non-model pre-filter remove a large fraction of a contract's source without changing the model's classification behaviour — and if so, how much of the token budget does that actually recover?

Two constraints followed from an early design review and shaped everything below. First, the filter must be **fully static**: every rule decidable by regular expressions and brace counting. A filter that itself requires model judgment to decide what to keep would consume the tokens it is meant to save. Second, overlapping evidence for different categories must be **emitted once**: eight independent per-category passes would send more text than the unsliced baseline, not less.

2 Dataset and baseline configuration

DIVE provides contract-level binary labels for 22,330 Solidity contracts across the first eight DASP Top-10 categories. Labels are **contract-level only** — the dataset carries no line numbers, no function names, and no localisation of any kind. This is a material constraint on the work: it means the pre-filter cannot be validated against known vulnerable lines, only against the weaker criterion that a label-positive contract retains *some* block tagged with that category.

Table 1. Sample composition. Contracts were drawn by `select_stratified.py` with `seed=20260827`: a seeded random draw, then repaired so every category has at least 5 positives and 5 negatives. This keeps base rates near DIVE's natural prevalence while making all eight categories measurable in both directions.

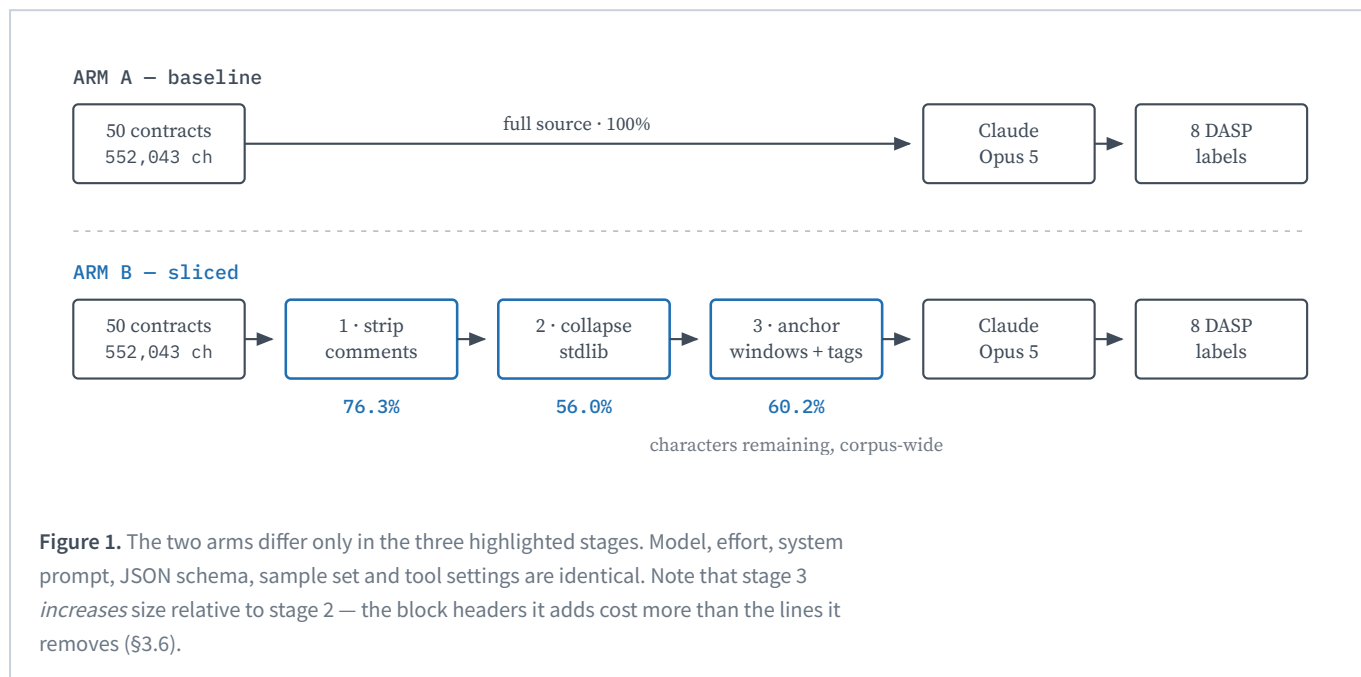
DASP CATEGORY	DIVE PREVALENCE	POSITIVES IN SAMPLE	SAMPLE RATE
Reentrancy	51.1%	22	44%
Access Control	74.9%	39	78%
Arithmetic	42.7%	23	46%
Unchecked Return Values	26.5%	5	10%
DoS	16.9%	7	14%
Bad Randomness	2.8%	5	10%
Front Running	2.7%	5	10%
Time manipulation	28.3%	13	26%
All categories	—	119 / 400	29.8%

The 50 contracts total 552,043 characters (min 2,358; median 11,484; max 27,687; none truncated). Compiler versions split 27 legacy ($\wedge 0.4$, $\wedge 0.5$) against 23 modern ($0.8+$) — a distribution that turns out to matter in §3.3. The baseline arm runs the Claude Code CLI in headless mode with tools disabled, so the model sees nothing but the prompt; a JSON schema forces a parseable label vector.

3 Method: the static pre-filter

3.1 Pipeline

The filter runs four stages over each source file. Stage 3 is the only stage that consults the DASP taxonomy; stages 1, 2 and 4 are taxonomy-independent.



3.2 Anchors and the single surviving gate

Each category owns a set of anchor regular expressions. A line matching an anchor marks its enclosing region as a candidate for that category. Two shared macros do most of the work: EXT matches any external or cross-contract call, and MUTFN matches a state-mutating public/external entry point.

Table 2. Final anchor set. Recall is measured against the DIVE labels over all 50 contracts and is 1.00 for every category.				
CATEGORY		ANCHORS	GATE	RECALL
Reentrancy	EXT		view/pure	22/22
Access Control	modifier · only\w+ · owner= · tx.origin · selfdestruct · delegatecall · require(msg.sender · transferOwnership · admin · MUTFN		view/pure	39/39
Arithmetic	[+-%/]= · ++ · -- · x[+-%/]y · ** · << >> · unchecked		—	23/23
Unchecked Return Values	EXT		view/pure	5/5
DoS	for(· while(· .push(· .length · require(· revert · assert(· EXT		view/pure	7/7
Bad Randomness	blockhash · block. {difficulty,coinbase,gaslimit,number,timestamp,prevrandao} · now · keccak256 · sha3 · sha256 · random · nonce · seed		—	5/5
Front Running	approve(· allowance · msg.value · tx.gasprice · price · bid · swap*(· slippage · amountOutMin · deadline · reserve · MUTFN		view/pure	5/5
Time manipulation	now · block.timestamp · block.number · N days hours minutes weeks · deadline · startTime · endTime · cooldown · lastClaim · timeout		—	13/13
All	—		—	119/119

Two anchors were added late, after failure analysis, and each recovered a category that pure syntactic matching missed:

- **XCALL — a call on a non-builtin receiver.** Contract 4704's reentrancy is `petCardData.setPetAuras(...):` a call through an interface-typed state variable, with no `.call`, `.send` or `.transfer` anywhere in the file. Matching `receiver.method(` while excluding builtins (`msg`, `block`, `tx`, `abi`, `SafeMath`, ...) recovers it.
- **MUTFN — every state-mutating entry point.** Contract 9837 is a plain fixed-supply token with no owner, no modifier and no access-control construct at all. Its Access Control label describes an *absence*, which no positive pattern can match. Treating every non-view public function as a candidate is the only static way to surface it.

3.3 Rules that were rejected, and why

The first design carried five semantic gates intended to suppress obvious false-positive regions. Each was ablated against the DIVE labels. Four had to be removed: they encoded correct security reasoning but disagreed with the ground truth we are being scored against. We record them because the negative result is the more informative one.

Table 3. Gate ablation. Only the `view`/pure gate survives. Recall is measured on the affected category.

PROPOSED GATE	RATIONALE	RECALL EFFECT	KEPT
<code>pragma ≥ 0.8</code> suppresses Arithmetic	<code>solc 0.8</code> checks arithmetic by default	1.00 → 0.43	no
Function writes no state ⇒ not Reentrancy	no state to leave inconsistent	0.91 → 0.36	no
<code>require(...send())</code> ⇒ return value checked	the result is not dropped	0.60 → 0.20	no
Constant-bound loop ⇒ not DoS	iteration count is not attacker-controlled	no change	no
<code>view</code> /pure body cannot mutate	no state change is possible	no loss	yes

FINDING — A PROPERTY OF THE LABELS, NOT THE FILTER

23 of the 50 contracts compile under `solc 0.8+`, where arithmetic reverts on overflow, and DIVE labels many of them Arithmetic-positive anyway. Similarly, DIVE marks contracts Reentrancy-positive whose only external call is a state-free token forward. The gates were not wrong about Solidity; they were wrong about DIVE. Any filter tuned to maximise agreement with this dataset inherits the dataset's annotation policy, and that limits how far these anchors generalise.

3.4 Windows, merging, and single emission

An anchored line expands to its innermost enclosing brace block, capped at 10 lines; beyond that cap it falls back to the anchor ± 2 lines. The enclosing function signature is always attached so visibility and modifiers stay visible. Windows are then sorted and merged: any two whose ranges overlap or abut are fused into one block whose tag set is the *union* of theirs. This is what satisfies the single-emission constraint from §1 — a region relevant to both Reentrancy and Arithmetic is sent once, carrying both tags, rather than twice.

Blocks are rendered with a header naming their source line range and candidate categories:

```
// --- BLOCK 4 | src L37-60 | candidates: Access Control, Arithmetic, Bad Randomness, DoS, Front Running,
Reentrancy, Time manipulation, Unchecked Return Values
```

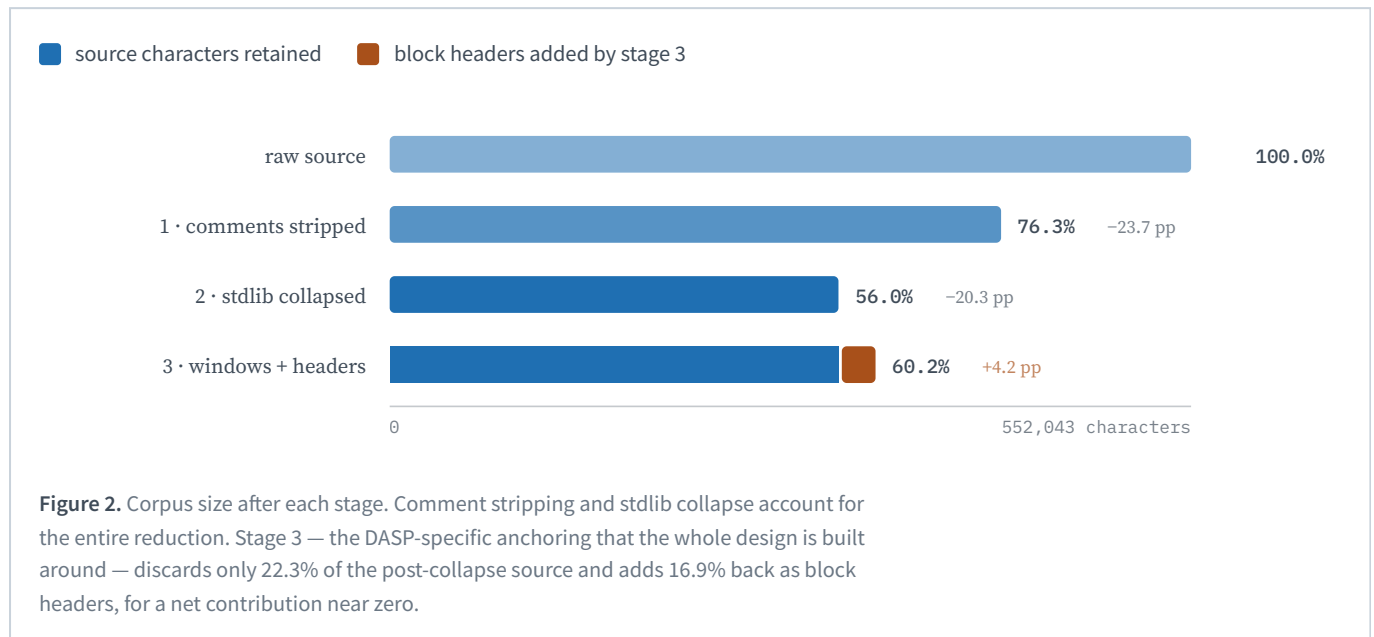
Averaged over the corpus this yields 14.9 blocks per contract.

3.5 Boilerplate collapse and the rescue pass

Unmodified standard-library constructs — `SafeMath`, `IERC20`, `Ownable`, `Context`, `IUniswap*` and relatives — are replaced by a one-line stub. Applied naively this cost recall: Access Control fell to 0.95 and DoS to 0.86, because in a handful of contracts the only anchor for a category sat inside a collapsed construct. A **rescue pass** repairs this: if a category ends with zero blocks anywhere in the file but an anchor for it exists inside collapsed text, a three-line window is emitted there. This restores recall to 1.00 at a cost of roughly 2 percentage points of size.

3.6 Where the reduction actually comes from

Decomposing the pipeline by stage produces the report's most consequential methodological finding, and it is not the one we expected.



FINDING — THE ANCHORS DO NOT SAVE TOKENS

Two of the three stages are taxonomy-independent and do all the size work. The anchor machinery — eight regex sets, brace-matched windows, merging, the rescue pass — contributes essentially no net reduction once its own block headers are paid for. Its output is therefore best understood not as compression but as **annotation**: it tells the model where to look. Whether that annotation helps is an empirical question, and §6 argues it currently hurts.

3.7 Two implementation defects found during validation

Both were silent — they produced plausible output that was wrong — and are recorded as a caution for anyone reimplementing this.

1. **Comment stripping corrupted brace matching.** The block-comment pattern `/\*..*?\*/` matched the `/*` embedded in the line comment `/** is exponent, price doubles every level and consumed everything up to the next */`, deleting a closing brace. A whole library `SafeMath` was consequently parsed as part of the contract above it and never collapsed. Fixed by tokenising strings and both comment forms in a single pass.
2. **Reported line numbers did not match the source file.** Block comments were removed rather than replaced by their newline count, so every `src L37-60` annotation after the first block comment was offset. Fixed by preserving line count when stripping.

4 Experimental design

Arm A is the pre-existing baseline (`results_50_calibrated.json`). Arm B adds a `--slice` flag to the same harness, which replaces each sample's source with the filter's output and prepends an explanation of the block format. Every other variable is held fixed: model opus, effort high, prompt variant calibrated, tools disabled, identical JSON schema, identical sample file, one call per contract.

The preamble given to Arm B explicitly instructs the model to treat tags as lexical hints rather than findings, states that most tagged blocks are not vulnerable, and notes that omitted code matched no anchor. This wording is itself part of the treatment; §7 discusses the confound it creates.

Because both arms classify the same 400 label cells, the comparison is **paired**. We report McNemar's test on discordant cells as the primary inferential statistic and a paired t -test on per-contract agreement as a secondary check.

5 Results

5.1 Token consumption and cost

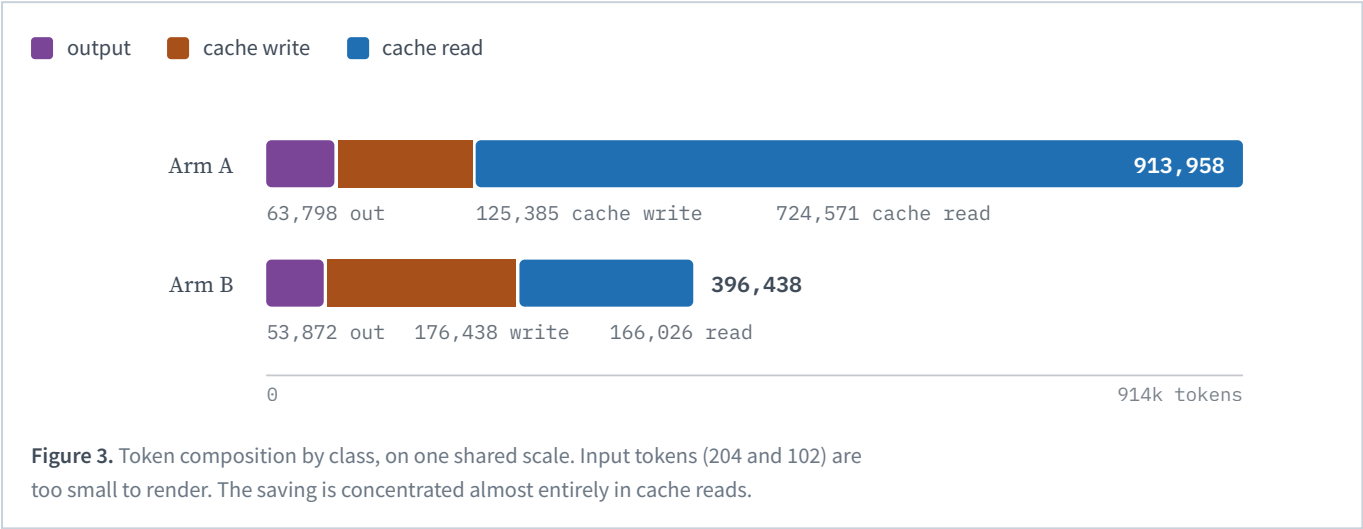


Table 4. Resource consumption across all 50 contracts.			
MEASURE	ARM A BASELINE	ARM B SLICED	Δ
Input tokens	204	102	-50.0%
Output tokens	63,798	53,872	-15.6%
Cache write	125,385	176,438	+40.7%
Cache read	724,571	166,026	-77.1%
Total tokens	913,958	396,438	-56.6%
Mean tokens / contract	18,279	7,928	-56.6%
Wall clock	1,019.6 s	818.5 s	-19.7%
Mean s / contract	20.4	16.4	-19.7%
Monetary cost	\$3.4103	\$3.3636	-1.4%

FINDING — A 57% TOKEN CUT IS WORTH 1.4%

Two effects cancel. Output tokens are the most expensive class and roughly half the bill, and they fell only 15.6%: reasoning length is driven by having eight categories to adjudicate, not by how much source there is to read. Meanwhile the cheap 724k cache-read block was replaced by a 176k cache-*write* block priced roughly an order of magnitude higher per token. Slicing changed *which* tokens are spent far more than how many. The defensible benefits are context headroom and 19.7% wall clock — not budget.

5.2 Classification accuracy

Table 5. Accuracy over all 400 label cells. Hamming loss is inverted — lower is better.

METRIC	ARM A	ARM B	Δ
Label agreement	61.8%	62.5%	+0.7 pp
Hamming loss	38.2%	37.5%	−0.8 pp
Exact match (8/8)	0 / 50	2 / 50	+2
Precision	38.5%	41.4%	+2.9 pp
Recall	47.9%	63.0%	+15.1 pp
F1	42.7%	50.0%	+7.3 pp
Cohen's κ	0.145	0.220	+0.075

Table 6. Confusion matrix over 400 cells. Every false negative recovered is matched by roughly one new false positive.

ARM	TP	FP	FN	TN
A — baseline	57	91	62	190
B — sliced	75	106	44	175
Δ	+18	+15	−18	−15

5.3 Significance

Of the 400 paired cells, 227 were correct in both arms and 130 wrong in both, leaving 43 discordant: 20 correct only under the baseline, 23 correct only under slicing.

Table 7. McNemar's exact test per category and overall. b = correct in A only, c = correct in B only. With eight comparisons a Bonferroni threshold is $\alpha = 0.00625$; no category clears it.

CATEGORY	B	C	P (EXACT, TWO-SIDED)
Reentrancy	1	3	0.625
Access Control	2	9	0.065
Arithmetic	5	4	1.000
Unchecked Return Values	2	2	1.000
DoS	8	1	0.039
Bad Randomness	1	0	1.000
Front Running	0	3	0.250
Time manipulation	1	1	1.000
All 400 cells	20	23	0.761

The paired t -test on per-contract agreement agrees: $t = 0.434$, $df = 49$, mean difference $+0.06$ of 8 labels, 95% CI $[-0.21, +0.33]$. The interval comfortably contains zero.

PRIMARY RESULT

There is **no statistically detectable accuracy difference** between the arms ($p = 0.761$). The correct claim is non-inferiority: removing 40% of the source characters and 56.6% of the tokens did not measurably change how well the model agrees with DIVE. The $+0.7$ pp agreement gain in Table 5 is noise and should not be reported as an improvement.

5.4 Per-category behaviour

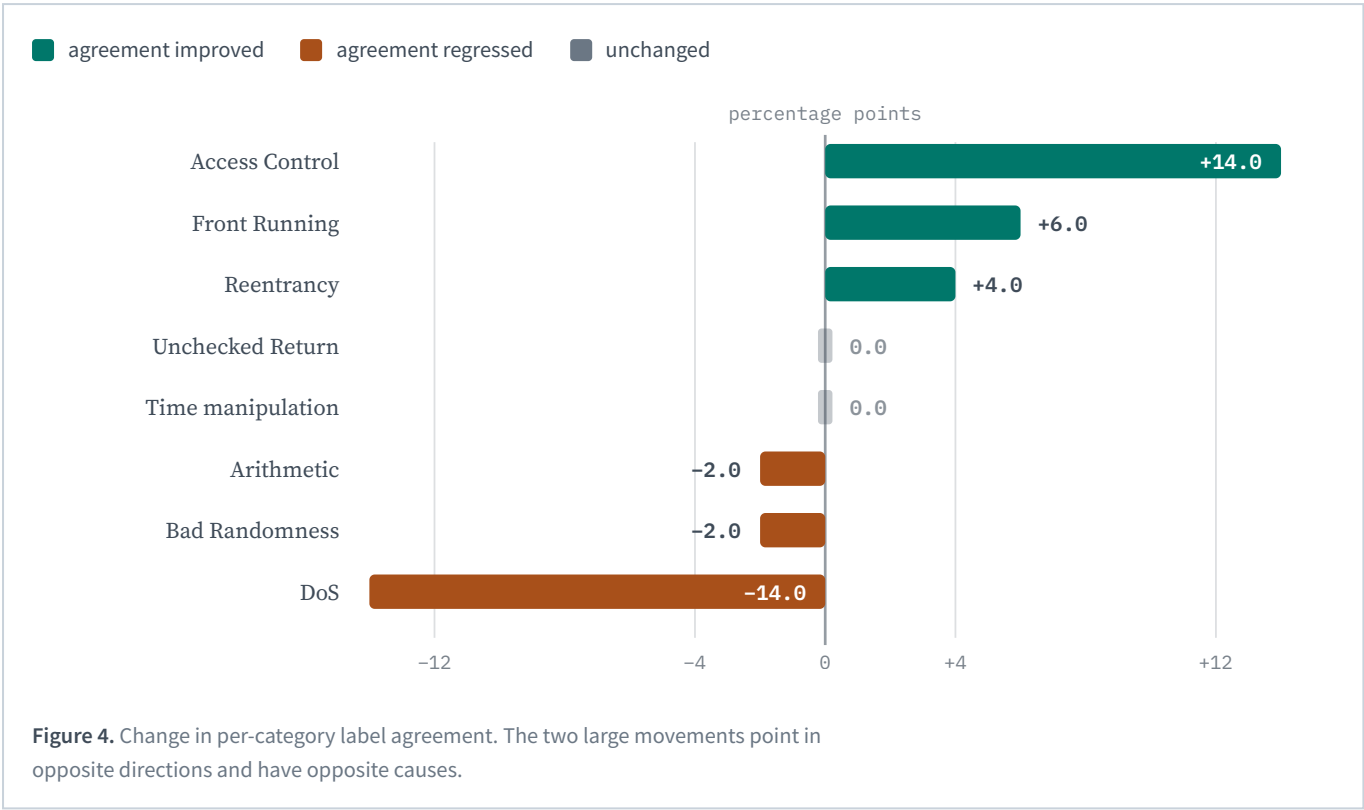


Table 8. Per-category detail. "LLM+" counts contracts the model marked positive; "DIVE+" counts ground-truth positives.

CATEGORY	DIVE+	LLM+ A	LLM+ B	AGREE A	AGREE B	Δ PP	FN A	FN B
Reentrancy	22	23	27	0.820	0.860	+4.0	4	1
Access Control	39	6	17	0.340	0.480	+14.0	33	24
Arithmetic	23	17	22	0.560	0.540	-2.0	14	12
Unchecked Return Values	5	12	16	0.700	0.700	0.0	4	2
DoS	7	13	20	0.760	0.620	-14.0	3	3
Bad Randomness	5	3	4	0.960	0.940	-2.0	2	2
Front Running	5	46	47	0.100	0.160	+6.0	2	0
Time manipulation	13	28	28	0.700	0.700	0.0	0	0
All	119	148	181	0.618	0.625	+0.7	62	44

5.5 Per-contract outcomes

14 contracts improved, 23 were unchanged, 13 regressed. The largest single regression is contract 5665, which fell 6/8 → 3/8. Its ground truth is all-zero except Time manipulation, and Arm B added three false positives — Unchecked Return, DoS and Bad Randomness — on precisely the three categories named in its block headers.

6 Analysis

6.1 The movement is a bias shift, not a capability gain

Arm B predicted 181 positives against Arm A's 148, where the ground truth has 119. Recall rose 15.1 points and precision 2.9, but Table 6 shows what paid for it: 18 recovered false negatives against 15 new false positives. Cohen's κ improved from 0.145 to 0.220 largely because Arm A was badly under-predicting — a base-rate correction, not better reasoning. Both arms remain in the "slight agreement" band; neither is a usable auditor at these numbers.

6.2 Access Control: the anchor worked as designed

Access Control is the most prevalent label in the sample (39/50) and the baseline's worst failure, catching only 6. The MUTFN anchor tags every state-mutating entry point — exactly the sites where a missing modifier would live — and Arm B found 11 more, cutting false negatives $33 \rightarrow 24$ for 2 new false positives. This is the one category where the annotation carried real information, and it is the strongest single argument for keeping tags in some form. Even so, the per-category test does not clear Bonferroni correction ($p = 0.065$).

6.3 DoS: an anchor that tags everything informs nothing

The DoS anchor includes `require()`, `revert` and `assert()`, which appear in nearly every non-trivial function, so almost every emitted block carries a DoS tag. The model read that ubiquity as evidence: positives went $13 \rightarrow 20$ against a ground truth of 7, while false negatives stayed at 3. The tag recovered nothing and cost 14 points of agreement — the largest per-category movement in the experiment ($b = 8$, $c = 1$).

Taken with contract 5665, this is a consistent mechanism: **the model treats the tags as weak evidence despite an explicit instruction not to**. A tag that fires on 90% of blocks carries no information but still shifts the prior.

6.4 What the token result does and does not license

The load-bearing result is negative and robust: discarding 40% of the characters changed nothing measurable, so the model was not using the discarded text. That holds independently of the tagging question and independently of the cost analysis. It licenses aggressive pre-filtering as a scaling strategy for the full 22,330-contract corpus, where the binding constraint is context and throughput rather than dollars.

7 Threats to validity

- **The treatment confounds two changes.** Arm B both removes code and adds category tags plus an explanatory preamble. §6.3 gives good reason to think these have opposite-signed effects, so the null result may be a cancellation rather than a true absence of effect. Disentangling them requires the third arm proposed in §8.
- **Single run, $n = 50$.** No repeated sampling and no variance estimate across runs. With a non-deterministic model, some of the 43 discordant cells are almost certainly run-to-run noise rather than treatment effect.

- **Filter recall is validated against a weak criterion.** DIVE has no line-level annotation, so "recall 1.00" means only that a label-positive contract retains *some* block tagged with that category — not that it retains the specific lines the annotator was looking at. The filter could in principle keep the wrong evidence and still score 1.00.
- **The anchors are tuned on the evaluation set.** Anchors and gates were developed by iterating against these same 50 contracts. The 1.00 recall figure is therefore an in-sample fit and will be optimistic on held-out data; §3.3 shows the tuning absorbed DIVE-specific annotation policy.
- **Absolute agreement is low in both arms.** At $\kappa \approx 0.15\text{--}0.22$ the model is barely above chance on this taxonomy. Comparisons between two weak classifiers are correspondingly fragile, and improvements in F1 should not be over-read.
- **Cost figures are platform-specific.** The 1.4% result depends on the cache-write/cache-read price ratio and on this harness's caching behaviour; a different serving configuration could turn the same token reduction into a substantially larger or smaller saving.

8 Conclusions and next experiments

A purely static pre-filter removed 40% of source characters and 56.6% of tokens from a DASP-8 classification task with no statistically detectable change in agreement ($p = 0.761$). Monetary saving was negligible (1.4%) because the reduction lands on the cheapest token class; wall clock fell 19.7%. The category tags the filter emits appear to bias the model toward positive predictions, helping where an anchor is selective (Access Control, +14 pp) and hurting where it is not (DoS, -14 pp).

Three experiments follow directly, in priority order.

1. **Run a tag-free third arm.** Same sliced source, block headers reduced to line ranges with no category names. This separates the two confounded treatments and, given §6.3, is the most likely configuration to beat both current arms. It also removes the 16.9% header overhead identified in §3.6, pushing output below 52% of source.
2. **Suppress tags above a coverage threshold.** If a category tags more than roughly 60% of a contract's blocks it is not discriminating and should be omitted for that contract. This is a two-line change and directly targets the DoS failure while preserving the Access Control gain.
3. **Re-tune the DoS anchor and re-validate.** Drop `require()`, `revert` and `assert()`, keeping loops and external calls, then re-check recall. If DoS recall survives at 7/7, most of the 14-point loss should recover without touching anything else.

Beyond these, the filter should be re-validated on a held-out DIVE split to obtain an out-of-sample recall estimate before any claim about the full corpus, and the sliced protocol should be repeated across several runs to establish run-to-run variance.

```
Reproduction. python3 select_stratified.py -n 50 --seed 20260827 python3 slice.py --self-check python3
run_eval.py --samples samples_50.json --out results_50_sliced.json --model opus --effort high --
prompt calibrated --slice python3 compare_ab.py results_50_calibrated.json results_50_sliced.json
```

Artifacts. `slice.py` (pre-filter), `compare_ab.py` (A/B statistics), `results_50_calibrated.json` (Arm A), `results_50_sliced.json` (Arm B), `sliced_50.json` (filter output for all 50 contracts).

All figures computed directly from the two result files. Statistical tests: McNemar exact binomial on paired label cells; paired *t*-test on per-contract agreement.